Gon AI Development OS · 2026년 11월 1일 · v1.2

# 이 매뉴얼로 시작하는 방법

AI와 문제를 탐색하고 근거를 조사해 방향을 정한 뒤, 필요한 개발·검증·전달로 이어간다.

> **Herdr에서 프로젝트 폴더와 기획 Codex를 준비한 뒤 P1 → P5를 진행하고, 개발을 선택하면 01 → 10으로 이어가세요.**
> 모든 단계를 매번 별도 채팅으로 요청할 필요는 없습니다. 전체 작업을 맡기면 AI가 허용된 범위에서 이어가고, 사용자는 목표·중요한 선택·실제 결과를 확인합니다.

**새 아이디어·프로젝트**
P1 문답 → 리서치·판단 → P5 기획 인계

**기존 프로젝트**
01에서 현재 코드·문서 확인 → 남은 단계

**중단 후 재개**
10의 재개 요청문 → 기록과 현재 상태 대조

**AEOS 프로젝트**
기존 .ai 계약 유지 → 일반 문서 중복 생성 금지

| 준비할 항목 | 사용자가 정할 내용 |
|---|---|
| **프로젝트** | 이름, 대상 사용자, 해결하려는 문제 |
| **작업 위치** | 새 폴더 또는 기존 저장소 경로. 기획 단계부터 같은 폴더의 docs/research에 저장 |
| **이번 도착점** | 설계만 / 구현·검증 / 배포 준비 / 실제 배포 중 어디까지인지 |
| **제약** | 기존 기술, 변경 금지 범위, 비용·데이터·외부 서비스 관련 조건 |

**첫 실행 요청문 · Codex에 붙여넣기**

```
$gon-project-lifecycle

[프로젝트 이름]을 시작하자.
목표: [누가 어떤 문제를 해결하는지]
대상: [실제 작업 폴더 또는 저장소]
이번 범위: [기획부터 구현·검증·배포 준비까지 등]
유지할 조건: [기술·데이터·변경 금지 범위]

현재 상태를 확인하고 이 프로젝트에 필요한 단계부터 진행해줘.
일반적인 가역 작업은 계속하고, 중요한 미정 사항만 확인해줘.
각 단계의 결과를 실제 파일·실행·검증 근거로 확인해줘.
웹 프로젝트라면 사용 가능한 gon-web-engineering 지침도 적용해줘.
새 주 모델과 협업 작업자를 열 때 Codex·Gemini는 --yolo, Claude·Opus·Fable은 --dangerously-skip-permissions를 넣어줘.
기존 문서가 있으면 그대로 이어 쓰고 마지막에 다음 행동을 남겨줘.
```

대괄호 안을 실제 값으로 바꾸세요. “배포 준비”는 운영 환경 변경을 포함하지 않습니다. 이미 실제 배포를 요청하고 대상·범위가 명확하다면 같은 권한을 단계마다 다시 묻지 않습니다.

**이 판은 P1부터 개발까지 Herdr + Codex CLI로 통일합니다.** 기획 탭 A는 주 기획, B는 선택적인 독립 조사, C는 파일 확인용 쉘입니다. 기획 P1~P5는 같은 세션을 유지하고, P5 확정 뒤 개발 탭의 새 Codex가 문서를 읽어 시작합니다.

# 전체 흐름과 네 번의 확인

문서가 생겼다는 사실보다 다음 작업을 시작할 조건이 갖춰졌는지를 확인한다.

**P1** 아이디어 문답 — 기획 A
› **P2** 조사 질문·범위 — 같은 기획 세션
› **P3** 근거 조사·확인 — 같은 기획 세션
› **P4** 비교·반론·검증 — 같은 기획 세션
› **P5** 기획 확정·인계 — 문서 → 새 개발 세션

P1 → P2 → P3 → P4 → P5. P4에서 근거가 부족하면 P3, 문제 정의가 바뀌면 P1로 돌아갑니다. 직접 개발 선택 시 P5 → 01로 연결합니다. 기존 도구 사용·보류를 선택하면 결정과 다음 행동을 기록하고 종료할 수 있습니다. 기획·범위가 이미 명확하거나 작은 수정이면 필요한 개발 단계로 바로 갑니다.

**01** 대상 확인 — 경로·상태·권한
› **02** 목표와 범위 — 사용자·MVP·성공 기준
› **03** 구조 설계 — 데이터·화면·오류
› **04** 작업 기반 — 문서·저장소·환경
› **05** 첫 작업 — 입출력·범위·검사

**06** 구현 — 작은 기능 완성
› **07** 실제 검증 — 실패 → 수정 → 재검사
› **08** 검토와 통합 — 리뷰·PR·CI
› **09** 배포와 확인 — 허용 범위·운영 흐름
› **10** 기록과 재개 — 증거·다음 행동

위쪽 01–05를 마친 뒤 아래쪽 06–10으로 이어갑니다. 기능이 여러 개라면 05–08을 반복합니다. 07에서 실패하면 06으로 돌아갑니다.

상호작용형 전체 흐름도 열기 ↗

| 확인 지점 | 판단할 질문 | 충족하지 못했다면 |
|---|---|---|
| **G1 · 02 이후** | 첫 사용자가 얻을 결과와 제외 범위가 분명한가? | 02에서 핵심 요구만 좁힌다. |
| **G2 · 05 이후** | 어떤 파일을 바꾸고 무엇을 검사할지 실행 가능한가? | 03–05에서 막힌 계약·환경만 정리한다. |
| **G3 · 08 이후** | 해당 변경의 검사·필수 리뷰·통합 결과가 확인됐는가? | 06–08로 돌아가 원인별로 수정한다. |
| **G4 · 09 이후** | 실제 배포 대상과 관련 사용자 흐름이 확인됐는가? | 준비/배포/운영 확인 중 현재 위치를 기록한다. |

> 이 네 지점은 증거를 확인하는 기준입니다. 네 번의 별도 사용자 승인이나 자동화된 강제 차단 장치가 설치됐다는 뜻은 아닙니다.

## 규모에 맞게 줄이는 경로

| 요청 | 실제 진행 |
|---|---|
| **오탈자·단순 설정·짧은 문서 수정** | 현재 상태 확인 → 수정 → 결과 확인. 전체 문서 세트나 외부 리뷰를 추가하지 않는다. |
| **재현 가능한 버그** | 01 → 05(재현 조건) → 06 → 07 → 08 → 10. 배포 요청이 있으면 09 포함. |
| **다중 파일 새 기능·구조 변경** | 관련 설계·계획·테스트를 선택하고 완료 직전에 외부 리뷰 한 번. |
| **조사·설계 문서만 요청** | 01–05 중 필요한 부분에서 종료. 제품 구현 완료로 표시하지 않는다. |

실행 매뉴얼

# Herdr의 탭·분할 터미널·세션·파일 구분

작업 방식, 사용하는 화면, 실행 장소, 모델을 각각 선택하면 인계와 권한 혼동이 줄어든다.

**화면**

**Herdr space / tab / pane**

프로젝트 묶음 / 작업 화면 / 분할 터미널

**대화**

**Codex CLI 세션**

같은 결과는 이어가기 · 독립된 작업은 새 대화

**저장**

**프로젝트 문서**

새 세션도 실제 파일을 읽어 인계

tmux에서 한 화면을 나누는 단위도 pane입니다. 이 안내는 이를 '분할 터미널'이라고 부릅니다. Files·Changes는 보조 패널이며 Codex 대화 자체가 아닙니다.

| 칸 | 기획 탭 | 개발 탭 |
|---|---|---|
| A | 주 기획 Codex · P1~P5 같은 세션 · 통합 문서 작성 | 새 개발 Codex · 문서 읽기 → 구현과 수정 |
| B | 선택적인 독립 조사 Codex · 지정 notes 파일만 작성 | 실행 서버용 쉘 · 로그 확인 |
| C | 파일 확인용 쉘 · pwd / ls / less | 검사·문서 확인용 쉘 |

설치본 Herdr app에서 Ctrl+Shift+O로 오른쪽을 나누고, 오른쪽 칸을 선택한 뒤 Ctrl+Shift+E로 아래를 나눕니다. Ctrl+Shift+Z는 선택 칸 확대·복귀, F2는 탭 이름 변경입니다. 개인 키가 다르면 Ctrl+Shift+P 명령 팔레트와 도움말로 확인합니다.

새 칸은 쉘입니다. codex를 실행해야 새 AI 대화가 시작됩니다. 자연어 요청은 Codex 입력칸, 쉘 명령은 Codex가 실행되지 않은 칸에 넣습니다. 같은 세션을 두 칸에서 동시에 재개하거나 같은 파일을 동시에 수정하지 않습니다.

# P1 질문에 답하고 문제 정의를 파일로 남긴다

AI의 질문·내 답변에서 이번 프로젝트의 문제와 제약을 찾습니다.

**열 창** Herdr · 기획 탭 · A 분할 터미널의 Codex

**세션** Herdr 준비 단계에서 시작한 '기획' 대화를 P1~P5까지 이어갑니다. 단계가 바뀌어도 /new를 입력하지 않습니다.

채팅은 대화 기록이고, Markdown(.md) 파일은 프로젝트 폴더에 남는 글 문서입니다. AI가 "정리했습니다"라고 말한 것만으로 저장된 것은 아닙니다.

### 저장·확인할 파일

- `docs/research/01-problem.md` · A 작성 · 질문과 답변·문제·제약·미정

### 사용자가 진행할 순서

1. 00에서 프로젝트 폴더와 기획 세션을 먼저 준비합니다. A를 클릭해 아래 요청문을 보냅니다.
2. 질문은 한 번에 1~3개씩 받습니다. 모르면 "미정", 추측이면 "가정"이라고 답합니다. AI가 임의로 내 답을 채우지 않게 합니다.
3. AI가 요약한 대상·불편·원하는 변화·제약을 읽고 잘못 이해한 부분을 바로잡습니다. 아직 기능이나 기술을 급하게 확정하지 않습니다.
4. 대화가 한 덩어리 끝날 때 01-problem.md에 실제 저장하도록 합니다. C의 파일 확인 명령 또는 Files 패널로 파일 내용을 엽니다.
5. 누가 어떤 상황에서 무엇 때문에 불편한지 한 문장으로 설명되면 P2로 갑니다. 답변이 부족하면 이 세션에서 계속 문답합니다.

| 정할 항목 | 내용 |
| --- | --- |
| **입력** | 한 문장 아이디어 + 실제 겪은 불편 |
| **저장** | docs/research/01-problem.md |
| **문서 상태** | 초안 → 사용자 확인 |
| **이번에 미뤄도 되는 것** | 세부 기술·전체 기능·이름 |

**C · 저장된 문제 정의 읽기 · C의 쉘에서 실행합니다. 파일이 없으면 P1 저장부터 확인합니다.**

```
cd '[실제 프로젝트 경로]'
sed -n '1,160p' docs/research/01-problem.md
```

**Herdr 기획 A의 Codex 대화 입력칸에 붙여 넣기**

```
[프로젝트 이름] 기획을 시작하자. 현재 프로젝트 폴더는 [실제 프로젝트 경로]야.
내 아이디어: [실제로 하고 싶은 일 한 문장으로 바꾸기]
실제로 불편했던 상황: [내 경험으로 바꾸기]
대상 사용자·현재 방법·불편·원하는 변화·시간/예산/환경 제약을 질문해줘.
한 번에 중요한 질문 1~3개만 하고 내 답을 기다려. 답하지 않은 내용을 지어내지 마.
문답이 한 덩어리 끝날 때마다 docs/research/01-problem.md를 갱신해줘.
질문/내 답변, 확인한 사실, 가정, 미정, 문제 한 문장, 다음 조사 질문을 구분해.
이 단계에서는 코드 구현과 도구 설치를 시작하지 마.
문서에는 확정 사실·사용자 결정·AI 제안·미확인을 구분해줘. 파일을 실제로 저장하고 다시 읽어 경로와 핵심 내용을 알려줘. 저장할 수 없으면 저장했다고 말하지 말고 이유와 본문을 보여줘. 기존 문서는 읽고 필요한 부분만 갱신해줘.
```

**파일이 실제로 저장됐는지 확인 · A · 같은 기획 세션**

지금까지 내가 답하고 정정한 내용으로 docs/research/01-problem.md를 갱신해줘. 저장한 파일을 다시 읽고 절대 경로·핵심 내용·아직 미정인 내용을 알려줘. 다음 단계는 시작하지 마.

**AI와 주고받는 예시 · 실제 조사 결과 아님**

나: 앱을 만들고 싶어.
AI: 누가 어떤 순간에 불편한가요?
나: 내가 연습용 할 일을 자주 잊어. 여러 기기 동기화는 필요 없어.
기록: “개인 학습용”은 결정, “다른 사람도 필요하다”는 아직 가정.

### 확인할 결과

- 01-problem.md를 열어 내 실제 답변과 다른 내용이 없는지 확인했다.
- 문제 한 문장·대상·제약·미정이 구분되어 있다.

### 다음 행동

같은 A 세션에서 P2. 기술을 정하지 못했어도 조사 질문이 명확하면 진행합니다.

**막히면** AI 답변만 보이고 파일이 없으면 아래 “저장 확인 요청”을 보냅니다. 범위가 너무 크면 사용자 한 명·장면 한 개로 좁힙니다.

# P2 질문별 조사 방법과 저장 담당을 정한다

“많이 조사” 대신 무엇을 확인하면 결정할 수 있는지 정합니다.

**열 창** Herdr · 기획 탭 · A 분할 터미널의 Codex

**세션** Herdr 준비 단계에서 시작한 ‘기획’ 대화를 P1~P5까지 이어갑니다. 단계가 바뀌어도 /new를 입력하지 않습니다.

분할 터미널을 하나 더 만들면 새 쉘이 생깁니다. Codex를 실행해야 두 번째 AI 대화가 시작됩니다. 두 AI는 파일을 공유할 수 있지만 서로의 채팅을 자동으로 읽지는 않습니다.

### 저장·확인할 파일

- `docs/research/01-problem.md` · A·B 읽기 · 조사 배경
- `docs/research/02-research-plan.md` · A 작성 · 질문·방법·담당·완료 기준
- `docs/research/notes/tool-comparison.md` · B만 작성(선택) · 독립 조사 결과

### 사용자가 진행할 순서

1. A에서 P1 파일을 읽게 하고 아래 요청문을 보냅니다. 조사 질문에 Q01, Q02처럼 번호를 붙입니다.
2. 각 질문의 이유, 찾아볼 공식 자료, 직접 확인할 방법, 그만 조사할 기준을 정합니다. 시간 범위도 적습니다.
3. 혼자 순서대로 진행하면 A만 사용합니다. 독립적인 조사 한 건이 있으면 B를 클릭하고 새 Codex를 시작합니다. A 세션을 B에서 resume하지 않습니다.
4. B의 이름을 “프로젝트-도구조사”처럼 구분하고, 읽을 파일과 결과를 쓸 파일 하나를 지정합니다. A와 B가 같은 통합 문서를 동시에 수정하지 않게 합니다.
5. 02-research-plan.md에서 질문·담당·결과 경로를 확인합니다. 질문이 흐리면 P1로 돌아갑니다.
6. 출력 품질이 핵심이면 대표 입력·독립 기대 결과·오류 또는 누락 기준을 정하고 UI 확장 전에 작은 실제 입력 검사를 배치합니다.

| 정할 항목 | 내용 |
| --- | --- |
| **입력** | 01-problem.md |
| **저장** | docs/research/02-research-plan.md |
| **A의 역할** | 문제·질문·통합 문서 관리 |
| **B의 역할(선택)** | 분리된 질문 조사 → notes/tool-comparison.md |

**B · 독립 조사용 새 Codex 시작 · 쉘에 붙여 넣습니다. Codex의 대화 입력칸에는 넣지 않습니다.**

```
cd '[실제 프로젝트 경로]'
codex --search --yolo
```

**B · 조사 세션 이름 붙이기 · Codex가 실행된 입력칸에 한 줄씩 입력합니다.**

```
/rename [프로젝트 이름]-도구조사
```

**Herdr 기획 A의 Codex 대화 입력칸에 붙여 넣기**

docs/research/01-problem.md를 읽고 조사 계획을 작성해줘.
아직 결정할 수 없는 질문마다 Q01부터 번호를 붙여줘.
표 열은 질문 / 이 답이 필요한 결정 / 찾아볼 1차 출처 / 직접 확인할 방법 / 조사 종료 기준 / 담당 / 결과 파일이야.
인터넷 자료로 답할 수 있는 것과 실제 사용자·환경 확인이 필요한 것을 구분해.
우선순위 높은 질문부터 최소 조사 범위와 시간 한도를 제안해.
A는 통합 문서를 담당하고, B에게 줄 독립 질문이 있다면 notes/tool-comparison.md 하나만 쓰도록 분리해.
내가 범위를 확인하면 docs/research/02-research-plan.md에 저장해. 아직 검색 결과를 만들어 넣지 마.
문서에는 확정 사실·사용자 결정·AI 제안·미확인을 구분해줘. 파일을 실제로 저장하고 다시 읽어 경로와 핵심 내용을 알려줘. 저장할 수 없으면 저장했다고 말하지 말고 이유와 본문을 보여줘. 기존 문서는 읽고 필요한 부분만 갱신해줘.

**B에 보낼 독립 조사 요청 · B · 새 조사 세션**

이 세션은 독립 조사 담당 B야. 다른 세션의 대화를 알고 있다고 가정하지 마.
docs/research/01-problem.md와 02-research-plan.md를 실제로 읽어줘.
담당 질문은 [P2에서 정한 Q번호·질문으로 교체]야. 현재 웹 검색으로 원문을 확인해.
결과는 docs/research/notes/tool-comparison.md에만 작성해. 통합 문서와 코드 파일은 수정하지 마.
각 항목에 제목·URL·게시/수정일·확인일·직접 확인한 내용·한계·관련 Q번호를 넣어.
접근하지 못한 자료는 미확인으로 표시하고, 완료 후 저장 경로와 남은 미확인을 알려줘.

**AI와 주고받는 예시 · 실제 조사 결과 아님**

Q01: 기존 도구로 해결할 수 있나? → 공식 기능 문서 확인.
Q02: 실제로 자주 불편한가? → 내가 사용한 기록 또는 사용자 확인.
Q03: 브라우저 저장이면 충분한가? → 제약과 데이터 유실 조건 확인.

### 확인할 결과

- 모든 중요한 질문에 답을 찾는 방법과 종료 기준이 있다.
- 병렬 조사 시 A·B의 작성 파일이 겹치지 않는다.

### 다음 행동

조사 범위를 확인한 뒤 P3. B를 사용하지 않으면 A가 같은 질문을 순서대로 조사합니다.

**막히면** B가 A의 대화를 아는 것처럼 답하면 P1·P2 파일을 실제로 읽게 합니다. 같은 파일을 수정했다면 작업을 잠시 멈추고 현재 내용을 확인한 뒤 A가 통합합니다.

# P3 조사 결과를 출처가 있는 근거표로 합친다

인터넷에서 찾은 말과 실제로 확인한 사실을 구분합니다.

**열 창** Herdr · 기획 탭 · A 분할 터미널의 Codex

**세션** Herdr 준비 단계에서 시작한 '기획' 대화를 P1~P5까지 이어갑니다. 단계가 바뀌어도 /new를 입력하지 않습니다.

근거표는 링크 모음이 아닙니다. "이 URL이 어느 질문의 어떤 결론을 뒷받침하는가"가 보여야 합니다. 다른 AI의 답변 자체를 1차 출처로 삼지 않습니다.

### 저장·확인할 파일

- `docs/research/notes/*.md` · B 작성·A 읽기 · 아직 통합 전 조사 메모
- `docs/research/03-evidence.md` · A 작성 · 확인한 출처·관찰·추론·빈틈

### 사용자가 진행할 순서

1. A 또는 B가 P2의 질문을 조사합니다. 검색 결과 제목만 믿지 말고 연결된 공식 원문을 확인하도록 합니다.
2. B를 썼다면 저장 완료 응답을 기다립니다. Herdr의 Done은 새 응답이 왔다는 신호이며 근거 검증 완료 판정은 아닙니다.
3. A에게 B 결과 파일과 직접 수집한 자료를 다시 읽게 합니다. 관련 없는 자료를 줄이고 S01, S02처럼 출처 번호를 붙입니다.
4. C 또는 Files에서 03-evidence.md를 엽니다. 중요한 URL을 외부 브라우저로 열어 날짜·문맥·실제 내용을 확인합니다.
5. 접근 제한·출처 충돌·자료 부족은 그대로 남깁니다. 조사 범위가 잘못됐으면 같은 A 세션에서 P2를 수정합니다.

| 정할 항목 | 내용 |
|---|---|
| **입력** | P2 조사 계획 + notes/* 조사 결과 |
| **저장** | docs/research/03-evidence.md |
| **출처 번호** | S01·S02… / 내 관찰은 U01… |
| **반드시 구분** | 원문 사실 / 해석 / 미확인 |

**Herdr 기획 A의 Codex 대화 입력칸에 붙여 넣기**

```
docs/research/02-research-plan.md와 완료된 docs/research/notes/ 파일을 읽어줘.
높은 우선순위 질문부터 현재 웹의 1차 출처를 직접 열어 확인하고 docs/research/03-evidence.md에 통합해줘.
출처표 열: S번호 / 제목·기관 / URL / 게시·수정일 / 확인일 / 원문에서 확인한 사실 / 관련 Q번호 / 한계.
날짜를 알 수 없으면 알 수 없음으로 쓰고, 읽지 못한 원문을 읽었다고 하지 마. 긴 원문 전체를 복사하지 말고 필요한 짧은 근거와 요약을 남겨.
내 실제 관찰은 U번호로 따로 기록하고, AI 추론·서로 충돌하는 근거·미확인 질문을 별도 구분해.
B의 결과도 검토 없이 확정 근거로 올리지 마. 주요 결론에 필요한 원문 확인 상태를 표시해.
내가 직접 열어 확인할 핵심 링크와 다음 단계에서 판단할 항목을 알려줘.
문서에는 확정 사실·사용자 결정·AI 제안·미확인을 구분해줘. 파일을 실제로 저장하고 다시 읽어 경로와 핵심 내용을 알려줘. 저장할 수 없으면 저장했다고 말하지 말고 이유와 본문을 보여줘. 기존 문서는 읽고 필요한 부분만 갱신해줘.
```

**AI와 주고받는 예시 · 실제 조사 결과 아님**

나: 이 자료는 2년 전 가격이네.
AI: 현재 공식 가격표를 다시 확인하고 확인일을 적겠습니다.
기록: 최신 여부가 확인되지 않은 가격은 "미확인"으로 남깁니다.

### 확인할 결과

- 핵심 결론에 연결된 출처·날짜·관련 질문이 있다.
- 확인 못 한 사실과 출처 충돌을 감추지 않았다.

### 다음 행동

근거가 충분하면 P4. 부족하면 질문을 좁혀 P2 또는 추가 조사로 돌아갑니다.

**막히면** 검색이 안 되면 /status와 실행 옵션을 확인합니다. 필요하면 A에서 기록을 저장한 뒤 /quit로 Codex를 종료합니다. 같은 A의 쉘에서 codex --yolo resume --search를 실행해 기존 기획 대화를 선택합니다. B·C에서 같은 세션을 동시에 재개하지 않습니다. 검색 가능과 로그인된 유료 자료 접근은 별개입니다.

# P4 대안을 비교하고 내 결정을 기록한다

개발할지, 기존 도구를 쓸지, 더 확인할지 판단합니다.

**열 창** Herdr · 기획 탭 · A 분할 터미널의 Codex

**세션** Herdr 준비 단계에서 시작한 '기획' 대화를 P1~P5까지 이어갑니다. 단계가 바뀌어도 /new를 입력하지 않습니다.

"AI 추천"과 "내가 선택"은 다릅니다. AI가 추천했다고 확정으로 쓰지 않고, 내 답변을 받은 뒤 결정 기록을 갱신합니다.

### 저장·확인할 파일

- `docs/research/03-evidence.md` · A 읽기 · 비교의 근거
- `docs/research/04-decision.md` · A 작성 · 결정·이유·제외·재검토 조건

### 사용자가 진행할 순서

1. A가 P1~P3를 읽고 기존 도구 사용·작게 직접 개발·보류를 비교하게 합니다. 비교 기준은 내 제약과 목표입니다.
2. 결정을 바꿀 중요한 가정 1~2개를 고릅니다. 직접 해 볼 수 있는 확인 방법과 통과 기준을 정합니다.
3. 사용자는 실제 사용·테스트·대화를 수행하거나 이번에는 못 했다고 알립니다. AI가 인터뷰나 수요 확인을 수행한 것처럼 쓰지 못하게 합니다.
4. 04-decision.md에 D01처럼 결정 번호를 붙이고 선택·이유·참조 S/U번호·제외 범위·미확인·다시 검토할 조건을 저장합니다.
5. 직접 개발을 선택하고 시작을 막는 미정이 해소되면 P5로 갑니다. 근거 부족은 P3, 문제 자체가 달라지면 P1, 기존 도구·보류는 결정 기록 후 마칩니다.

| 정할 항목 | 내용 |
|---|---|
| **입력** | 문제·조사 계획·근거표 |
| **저장** | docs/research/04-decision.md |
| **진행 선택** | 직접 개발 / 추가 확인 / 기존 도구 / 보류 |
| **검증 기록** | 누가·언제·무엇을 실제로 했는지 |

**Herdr 기획 A의 Codex 대화 입력칸에 붙여 넣기**

```
docs/research/01-problem.md, 02-research-plan.md, 03-evidence.md를 읽고 선택지를 비교해줘.
기존 도구 사용 / 최소 범위 직접 개발 / 보류를 내 목표·시간·비용·운영 부담으로 비교해.
중요한 주장은 S번호 또는 U번호와 연결하고 부족한 근거를 표시해.
결정을 바꿀 가정 1~2개, 직접 검증 방법, 통과 기준을 제안한 뒤 내 실제 확인 결과를 기다려.
직접 확인하지 않은 인터뷰·사용·수요·실험을 지어내지 마.
내 선택을 받으면 docs/research/04-decision.md에 D번호·선택·이유·근거·제외 범위·미확인·재검토 조건을 저장해.
학습 목적의 실습이라면 시장 수요 검증과 구분해. 아직 최종 기획 승인이나 구현 완료로 표시하지 마.
문서에는 확정 사실·사용자 결정·AI 제안·미확인을 구분해줘. 파일을 실제로 저장하고 다시 읽어 경로와 핵심 내용을 알려줘. 저장할 수 없으면 저장했다고 말하지 말고 이유와 본문을 보여줘. 기존 문서는 읽고 필요한 부분만 갱신해줘.
```

**AI와 주고받는 예시 · 실제 조사 결과 아님**

나: 기존 앱이 더 편하지만 React를 배우려는 목적이라 작은 앱을 만들래.
기록 D01: 학습용으로 직접 개발. 판매·사용자 수요는 검증하지 않았음.
제외: 로그인·동기화·상용 운영.

### 확인할 결과

- 04-decision.md의 결정이 내 선택과 일치한다.
- 실제 검증·가정·미확인과 재검토 조건이 구분되어 있다.

### 다음 행동

개발 선택 → P5 / 근거 부족 → P3 / 문제 재정의 → P1 / 기존 도구·보류 → 기록 후 종료.

**막히면** 큰 미정이 남았으면 승인 표시 대신 보류합니다. 개인 학습용 프로젝트와 사용자 수요를 검증한 제품을 구분합니다.

**개발하지 않을 때 · 결정 기록**

내 결정은 [기존 도구 사용 또는 보류]야. 이유와 실제 다음 행동·담당·재검토 날짜 또는 조건을 docs/research/04-decision.md에 저장해줘. docs/STATE.md에도 개발하지 않고 마친 상태를 연결해. 지금은 개발 인계 문서나 코드를 만들지 마.

# P5 자료를 개발 시작 문서 하나로 정리한다

새 개발 세션이 대화 전체를 몰라도 무엇을 만들지 이해하게 합니다.

**열 창** Herdr · 기획 탭 · A 분할 터미널의 Codex

**세션** Herdr 준비 단계에서 시작한 '기획' 대화를 P1~P5까지 이어갑니다. 단계가 바뀌어도 /new를 입력하지 않습니다.

최종 문서는 "무엇을 왜 만들고, 무엇이면 끝인지"를 정합니다. 세부 코드 설계는 개발 시작 후 구체화할 수 있습니다. 기술 미정 자체는 가능하지만 예산·데이터·필수 기능을 결정할 수 없는 미정은 먼저 풀어야 합니다.

### 저장·확인할 파일

- `docs/research/01~04` 문서 · A 읽기 · 앞 단계의 사실·근거·결정
- `docs/PROJECT-BRIEF.md` · A 작성 → 사용자 확인 · 개발 시작 때 읽을 최종 입구
- `docs/STATE.md` · A 갱신 · 현재 단계·세션 이름·다음 행동

### 사용자가 진행할 순서

1. A의 같은 기획 세션에서 P1~P4 파일을 모두 읽고 최종 문서 초안을 만듭니다. 채팅 전문을 이어 붙이는 작업이 아닙니다.
2. docs/PROJECT-BRIEF.md에 목표·사용자·첫 범위·제외·사용 흐름·요구사항·완료 기준·제약·미정·첫 작업을 정리합니다. 주요 결정은 D번호, 근거는 S/U번호로 연결합니다.
3. C에서 최종 문서를 직접 읽습니다. "데이터 저장"처럼 모호한 표현은 "브라우저 새로고침 후 같은 목록이 남음"처럼 확인 가능한 기준으로 고칩니다.
4. 초안을 읽고 수정할 부분을 A에 답합니다. 실제로 합의한 뒤 아래 승인 기록 요청을 보냅니다. 코드로 들어가기 전에 막는 미정이 남으면 보류로 둡니다.
5. 확정한 문서에는 버전·실제 승인일·변경 요약을 기록합니다. docs/STATE.md에는 최종 문서 경로, 기획 세션 이름, 남은 미정과 다음 첫 요청을 적습니다.
6. 이제 기획 탭은 보관하고 개발 탭에서 새 세션을 시작합니다. 새 세션은 최종 문서를 읽어 요약부터 하며, 곧바로 구현하지 않습니다.

| 정할 항목 | 내용 |
| --- | --- |
| **최종 입구** | docs/PROJECT-BRIEF.md |
| **근거 보관** | docs/research/01~04 문서 + notes/ |
| **상태** | 초안 → 사용자 확인 → 개발 시작 승인 또는 보류 |
| **다음 세션** | 개발 탭의 새 Codex 대화 |

**C · 최종 문서 열어 읽기 · 방향키로 읽고 q를 누르면 쉘로 돌아옵니다.**

```
cd '[실제 프로젝트 경로]'
less docs/PROJECT-BRIEF.md
```

**Herdr 기획 A의 Codex 대화 입력칸에 붙여 넣기**

docs/research/01-problem.md, 02-research-plan.md, 03-evidence.md, 04-decision.md를 실제로 읽어줘.
서로 충돌하는 결정이나 시작을 막는 미정이 있으면 먼저 나에게 확인해.
개발 세션이 처음 읽을 docs/PROJECT-BRIEF.md 초안을 작성해줘.
포함할 내용: 문서 상태·버전·날짜, 한 줄 목표, 사용자와 문제, 첫 출시 포함/제외, 핵심 사용 흐름, 요구사항 R번호, 관찰 가능한 완료 기준 AC번호, 데이터·제약, 기술 확정/미정, 근거와 결정, 남은 미정, 첫 구현 작업.
요구사항은 결정 D번호와 연결하고, 결정은 근거 S/U번호로 연결해. 모르는 내용은 제안 또는 미정으로 남겨.
한 페이지 요약을 맨 앞에 두고 필요한 세부표를 뒤에 붙여. 조사 원문과 notes는 보존하고 상대 링크로 연결해.
초안 상태로 저장한 뒤 내가 읽을 핵심 부분과 승인 전에 답할 질문을 보여줘. 내 확인 없이 승인됨으로 바꾸지 마.
문서에는 확정 사실·사용자 결정·AI 제안·미확인을 구분해줘. 파일을 실제로 저장하고 다시 읽어 경로와 핵심 내용을 알려줘. 저장할 수 없으면 저장했다고 말하지 말고 이유와 본문을 보여줘. 기존 문서는 읽고 필요한 부분만 갱신해줘.

**초안을 읽고 합의한 뒤에만 보내기 · A · 같은 기획 세션**

최종 문서를 읽었고, 아래 내용에 합의했어.
수정/확정 사항: [내가 실제로 확인한 내용으로 교체]
시작을 막는 미정이 남았는지 다시 확인해줘. 없으면 docs/PROJECT-BRIEF.md에 실제 승인일·버전·확정 범위와 변경 요약을 기록해.
남았으면 승인 대신 보류로 기록해.
docs/STATE.md에 최종 문서 경로·기획 세션 이름·남은 미정·개발 세션 첫 요청을 저장해. 세션 ID는 확인할 수 있을 때만 적고 추측하지 마.
기획 원본은 남겨 두고 지금은 코드를 구현하지 마.

**AI와 주고받는 예시 · 실제 조사 결과 아님**

초안: “할 일을 저장한다.”
내 정정: “같은 브라우저에서 새로고침하면 목록과 완료 상태가 유지돼야 해. 기기 간 동기화는 제외.”
최종 R04 → AC04: 새로고침 후 목록·완료 상태 일치. 근거는 D02·U02·U03.

## 확인할 결과

- 최종 문서만 읽어도 포함·제외·완료 기준·미정·첫 작업을 알 수 있다.
- 내가 확인한 내용만 승인으로 기록했고 P1~P4 근거 링크가 열린다.
- STATE.md에 최종 문서 경로와 다음 개발 세션의 첫 요청이 있다.
- 외부 환경이 막힌 수용 기준과 그 기준에 실제로 의존하는 작업을 구분하고, 독립적으로 허용된 첫 작업을 정했다.

## 다음 행동

관리 매뉴얼 01에서 개발용 새 세션을 만들고 02에 문서 경로를 전달해 인계 확인합니다. 초보자 안내에서는 01→02→03입니다.

**막히면** 자료가 많으면 최종 문서의 앞쪽 요약을 줄이고 상세 근거를 링크합니다. 모순이 발견되면 P4를 먼저 수정한 뒤 최종 문서를 다시 생성합니다.

# 01 작업 대상과 시작점을 확인한다

이번에 어디에서 무엇을 바꾸고 어디까지 진행할지 한 문장으로 정한다.

**새 개발 세션:** P5의 확정 문서가 있으면 Herdr에서 Ctrl+Shift+T로 개발 탭을 만들고 같은 프로젝트 폴더로 이동한 뒤 codex --yolo를 실행합니다. 기획 탭은 남겨둡니다. 개발 A는 Codex, B는 서버, C는 검사·파일 확인용 쉘입니다. 기존 작업의 재개라면 새 대화 대신 codex --yolo resume로 선택합니다.

**작업 위치** Herdr의 기획 또는 개발 탭 · 해당 프로젝트 폴더의 Codex CLI

### 사용자가 준비할 것

프로젝트 이름·경로·기존 자료와 이번 도착점을 알려준다. 새 기획이라 경로가 아직 없으면 “기획부터, 저장소는 미정”이라고 적으면 된다.

### AI가 수행할 순서

1. 신규·기존·재개 중 현재 상황을 구분한다. 설명만 요청했는지 실제 변경을 요청했는지도 확인한다.
2. 기존 프로젝트는 실제 저장소 루트, 브랜치, 미커밋 변경과 적용 중인 AGENTS.md·README를 읽는다. 기존 CLAUDE.md가 있다면 충돌 여부도 확인한다.
3. 사용 가능한 스킬·도구·파일 접근을 확인한다. Local과 Cloud, 현재 폴더와 worktree를 구분한다.
4. 이미 허용된 작업을 파악하고, 구현을 막는 불명확한 대상만 질문한다. 미커밋 변경을 보존한다.

### 받아야 할 결과

- 작업 대상 경로와 현재 상태의 짧은 요약
- 이번 완료 조건 한 문장
- 지금 시작할 단계와 중요한 미정 사항

### 다음 단계로 가는 기준

작업 대상과 변경 범위가 명확하고 기존 변경을 보존할 방법을 알면 02 또는 남은 단계로 간다.

**그대로 사용할 요청문**

```
$gon-project-lifecycle
이 프로젝트의 실제 경로, 현재 브랜치, 미커밋 변경,
기존 지침·문서·검사 명령과 사용 가능한 스킬을 확인해줘.
이번 목표는 [목표], 도착점은 [구현·검증/배포 준비/실제 배포]야.
완료 조건을 한 문장으로 정하고 필요한 단계부터 이어가줘.
```

**예시**

“시험지 OCR 서비스의 기존 업로드 화면에 재시도를 추가하고, 로컬 정상·실패 흐름 확인까지 진행”처럼 범위를 정한다.

**막히면** 폴더를 읽지 못하면 접근 가능한 원문·명세로 독립 작업을 진행하고, 실제 파일 작업에 필요한 경로 또는 연결만 요청한다.

# 02 목표와 첫 출시 범위를 정한다

누가 어떤 입력으로 어떤 결과를 얻는지를 먼저 정한다.

**기획 인계:** docs/PROJECT-BRIEF.md와 docs/STATE.md를 실제로 읽히고 승인 상태·목표·포함/제외·완료 기준·미정·첫 작업을 요약해 확인합니다. 이미 확정된 결정을 임의로 다시 설계하지 않습니다.

> P1~P5를 진행했다면 기획 문서와 근거를 읽고 확정·제안·미정을 확인한다. 아래 예시보다 실제 결정을 우선하며 조사나 문서를 중복 생성하지 않는다. 문제·수요·대안이 불명확하면 P1 문답·리서치로 돌아간다.

**작업 위치** Herdr 개발 A의 같은 Codex 세션

### 사용자가 준비할 것

대상 사용자, 가장 불편한 문제, 제공할 입력·원하는 결과를 설명한다. “반드시 필요한 것”과 “나중에 해도 되는 것”을 구분한다.

### AI가 수행할 순서

1. 사용자 문제를 한 문장으로 정리하고 대표 사용 흐름을 3–5개 행동으로 적는다.
2. 첫 출시의 최소 유효 범위(MVP)와 제외 범위를 나눈다. 미래 요구를 현재 기능에 섞지 않는다.
3. 수용 기준을 관찰 가능한 입력·행동·결과로 쓴다. 정확도·성능 같은 목표는 측정 대상과 방법까지 정한다.
4. 확정 결정·제안·미정을 구분한다. 구현에 당장 필요하지 않은 미정은 이유와 다음 확인 시점을 남긴다.

### 받아야 할 결과

- PRODUCT 또는 기존 요구사항 문서
- 기능별 수용 기준
- 비범위·미정 목록

### 다음 단계로 가는 기준

첫 사용 흐름과 성공·실패 기준을 구체적으로 설명할 수 있으면 G1 통과. 세부 미래 기능까지 확정할 필요는 없다.

**그대로 사용할 요청문**

```
[아이디어]를 첫 출시 가능한 범위로 정리해줘.
대상 사용자, 문제, 입력과 출력, 대표 사용 흐름,
이번 범위와 제외 범위, 관찰 가능한 수용 기준을 작성해줘.
확정·제안·미정을 구분하고, 개발을 막는 질문만 먼저 확인해줘.
```

**예시**

“사진 1장 업로드 → 인식한 텍스트 확인 → 사용자가 수정 → 저장”을 첫 범위로 잡는다. 해설 생성·결제·대량 처리는 별도 작업으로 미룰 수 있다. 이는 설명용 예시이며 실제 프로젝트 결정은 요구에 맞춰 정한다.

**막히면** “좋게”, “빠르게”, “정확하게”만 있으면 대표 입력과 기대 결과를 추가한다. 데이터 없이 정확도 목표를 달성했다고 가정하지 않는다.

# 03 구조와 상태 흐름을 설계한다

작은 기능 하나가 화면·서버·데이터를 통과하는 경로와 실패했을 때의 동작을 정한다.

**작업 위치** Herdr 개발 A의 Codex CLI · 웹 구현이면 gon-web-engineering 관련 지침

### 사용자가 준비할 것

유지해야 할 기술·외부 연동·운영 제약을 알려준다. 기존 프로젝트라면 현재 API·DB·인증 방식이 기준이다.

### AI가 수행할 순서

1. 관련 코드와 기존 구조를 읽고 변경할 경계를 찾는다. 새 프로젝트의 기술은 실제 요구를 기준으로 선택한다.
2. 입력, 검증, 처리, 저장, 결과 반환을 연결하고 API·데이터 구조의 책임을 구분한다.
3. 빈 상태·대기·처리 중·성공·실패·재시도와 새로고침 후 복구를 필요한 만큼 설계한다.
4. 중요한 선택의 이유와 대안을 짧게 기록한다. 구조 결정이 바뀔 때만 결정 기록(ADR)을 남긴다.

### 받아야 할 결과

- ARCHITECTURE 또는 기존 구조 문서
- 화면·API·데이터의 계약과 상태 흐름
- 확정 기술과 남은 기술적 미정

### 다음 단계로 가는 기준

첫 기능의 입출력과 주요 오류 처리 방식이 정해지면 04로 간다. 전체 제품을 미리 세밀하게 설계할 필요는 없다.

**그대로 사용할 요청문**

```
확정한 MVP와 현재 저장소를 기준으로 첫 기능의 구조를 설계해줘.
화면·API·데이터 경계, 정상 흐름과 오류·재시도 흐름을 포함해줘.
기존 기술과 계약은 보존하고 변경이 필요한 이유를 기록해줘.
웹 작업이면 gon-web-engineering의 해당 지침을 적용해줘.
```

**예시**

신규 웹에서 기술이 미정이면 현재 웹 스킬은 React/TypeScript + Rust(Axum/Tokio) + PostgreSQL(SQLx)를 기본안으로 둔다. 기존 C#·Python 프로젝트를 이 조합으로 자동 변환하지 않는다.

**막히면** 외부 API나 인증 조건이 불명확하면 관련 계약을 먼저 확인한다. 조사만으로 접근 권한이나 실제 호출 성공을 확보했다고 기록하지 않는다.

# 04 문서와 실행 환경을 연결한다

다음 채팅에서도 같은 기준을 읽고 같은 검사를 실행할 수 있게 만든다.

**작업 위치** 실제 프로젝트 폴더를 연 Codex · 기준 GitHub 저장소

GitHub 저장소를 관리 기록의 기준으로 삼고 기능별 적용·불가·대체 기록을 관리 문서에 남긴다. 로컬 준비만 허용되거나 접근할 수 없으면 pending_sync로 구분한다.

### 사용자가 준비할 것

저장할 위치를 정한다. 새 원격 저장소 생성이 필요한 경우 이름과 공개 범위를 명확히 한다. 기존 프로젝트는 기존 문서·CI를 유지한다.

### AI가 수행할 순서

1. 일반 프로젝트와 AEOS 구조 중 현재 상황에 맞는 한 가지를 선택한다. 같은 내용을 두 문서 체계에 중복 작성하지 않는다.
2. 기존 문서가 없고 필요할 때만 AGENTS·README·PRODUCT·ARCHITECTURE·PLAN·STATE와 GitHub 관리 매핑을 구성한다. AI가 실제 내용과 명령을 채운다.
3. 매니페스트·잠금 파일·CI에서 실제 설치·실행·검사 방법을 확인하고 실행 가능한 최소 환경을 만든다.
4. 착수 전 검사 결과를 기록해 기존 실패와 새 회귀를 구분한다. 비밀값 대신 안전한 설정 방법만 문서화한다.

### 받아야 할 결과

- 문서 진입점과 실제 프로젝트 기준 문서
- 실행·검사 방법과 초기 결과
- 첫 작업의 선행 환경 조건

### 다음 단계로 가는 기준

다음 작업자가 문서 위치를 알고 필요한 명령을 실행할 수 있으면 05로 간다. 틀 생성만으로 내용 구성이 완료되지는 않는다.

**그대로 사용할 요청문**

```
이 프로젝트의 기존 문서와 개발 환경을 기준으로 작업 기반을 정리해줘.
같은 역할의 문서를 중복 생성하지 말고 필요한 내용만 보완해줘.
실제 설치·실행·검사 명령을 확인하고 기준선 결과를 기록해줘.
앱 구현까지 요청된 범위이므로 문서 정리 후 첫 기능으로 이어가줘.
```

**예시**

일반 프로젝트는 docs/PRODUCT.md 등에 기록한다. .ai/contracts·.ai/tasks·scripts/aeos.py가 보이면 AEOS 도입 상태를 확인하고 기존 체계를 이어 쓴다.

**막히면** 문서 생성기가 기존 파일을 감지하면 내용을 읽어 직접 연결한다. 경로 충돌·심볼릭 링크는 의도한 물리 경로를 확인한다. 설치되지 않은 명령을 실행 가능한 것으로 적지 않는다.

# 05 첫 작업을 검증 가능한 크기로 나눈다

한 번의 변경으로 관찰할 수 있는 결과를 정하고 구현 범위를 고정한다.

**작업 위치** Codex · GitHub Issues/Projects · AEOS 계약 링크

기존 이슈를 재사용하고 Project 상태·담당·라벨·Milestone을 실제 기준에 맞춰 연결한다. 업무 분해는 sub-issues, 선행 차단은 dependencies로 관리하며 지원 불가면 이슈 링크로 대체한다.

### 사용자가 준비할 것

가장 먼저 확인하고 싶은 사용자 행동을 고른다. AI가 제안한 순서에서 목표를 바꿀 필요가 있는지만 판단하면 된다.

### AI가 수행할 순서

1. 첫 기능의 입력·출력·허용 변경·제외 범위를 명시한다.
2. 선행 작업과 완료 조건을 적는다. 검증 방법이 없는 조건은 관찰 가능한 기준으로 바꾼다.
3. 버그는 수정 전에 실패 조건을 재현하고 정상 동작 기준을 기록한다.
4. 독립 작업의 위임이 명시적으로 요청되거나 적용 지침이 요구하는 경우에만 소유 파일과 통합 담당자를 정한다. 기본은 현재 AI가 직접 수행한다.

### 받아야 할 결과

- 구체적인 첫 작업과 검사 방법
- 관련 파일·자료·기준 revision
- 작업 순서와 제외 범위

### 다음 단계로 가는 기준

무엇을 바꾸고 무엇을 관찰하면 끝인지 분명하면 G2 통과. 구현을 요청했다면 여기서 멈추지 않고 06으로 간다.

**그대로 사용할 요청문**

```
첫 작업을 [사용자 행동]으로 잡고 구현해줘.
입출력, 허용 변경, 제외 범위와 수용 기준을 먼저 짧게 정리해줘.
각 기준을 확인할 검사 방법을 연결하고 관련 범위만 수정해줘.
버그라면 수정 전 실패 조건부터 확인해줘.
```

**예시**

“업로드 화면만 만들기”보다 “허용된 사진을 업로드하면 저장된 결과를 조회하고, 잘못된 형식은 거부하기”가 연결 상태를 검증하기 쉽다.

**막히면** 한 작업이 여러 독립 사용자 목적을 담고 있으면 나눈다. 파일 수가 적다는 이유만으로 작은 작업이라고 판단하지 않는다.

# 06 작은 기능을 끝까지 구현한다

사용자 행동 하나가 실제 결과로 이어지도록 필요한 구성만 구현한다.

**작업 위치** Herdr 개발 A의 Codex CLI · 같은 실제 폴더와 브랜치

### 사용자가 준비할 것

구현 중 새 요구를 추가할 때는 “이번 범위에 포함”인지 “후속 작업”인지 알려준다. 사소한 구현 선택은 AI가 기존 관례에 맞춰 진행한다.

### AI가 수행할 순서

1. 현재 코드의 진입점과 관련 테스트를 읽고 가장 작은 변경으로 기능을 연결한다.
2. 프론트엔드·API·저장을 필요한 만큼 함께 구현한다. 임시 화면이나 모의 데이터와 실제 연동을 구분한다.
3. 기존 스타일·패키지 관리자·API 계약을 유지하고 무관한 정리나 리팩터링을 섞지 않는다.
4. 작업 중 의미 있는 검사를 실행하고 실패 원인을 좁힌다. 도착한 결과와 다음 확인을 상태에 반영한다.

### 받아야 할 결과

- 요청과 직접 관련된 코드 변경
- 실행 가능한 첫 사용자 흐름
- 모의 처리·미연결·남은 항목의 구분

### 다음 단계로 가는 기준

구현된 흐름이 검증 가능한 상태이면 07로 간다. 코드 작성이나 프로세스 시작만으로 기능 완료라고 하지 않는다.

**그대로 사용할 요청문**

```
정리한 첫 작업을 구현해줘.
관련 코드·규칙을 먼저 확인하고 기존 변경을 보존해줘.
사용자 행동이 API와 저장 결과까지 이어지게 만들고,
임시 처리나 아직 연결되지 않은 부분은 명확히 표시해줘.
구현 후 관련 검증으로 계속 진행해줘.
```

**예시**

업로드 버튼 → 서버의 파일 형식 검사 → 저장 → 결과 조회를 연결한다. 업로드 성공 문구만 표시하고 저장이 안 되는 상태는 완료가 아니다.

**막히면** 동일 원인이 반복되면 가설을 바꾸고 관찰 근거를 추가한다. 의존 서비스가 막혔으면 가능한 독립 작업을 진행하고 차단 조건을 남긴다. 지연·할당량 때문에 모델을 임의로 바꾸지 않는다.

# 07 실제 동작을 검증하고 수정한다

수용 기준마다 실제로 무엇을 확인했는지 증거를 남긴다.

**작업 위치** Codex · 테스트 도구 · 필요한 브라우저·실제 연동 환경

### 사용자가 준비할 것

대표 입력·실패 사례·접근 가능한 시험 환경을 제공한다. 로그인된 사용자 흐름이 필요하면 안전한 인증 경로를 사용한다.

### AI가 수행할 순서

1. 변경에 맞는 빌드·타입 검사·테스트를 실행한다. 명령은 프로젝트와 CI의 실제 구성을 따른다.
2. 정상·빈 입력·잘못된 입력·중복/재시도 등 요구와 관련된 경계를 검증한다.
3. UI 변경은 관련 실제 사용자 흐름과 필요한 화면 크기·오류 상태를 확인한다. 적절한 연결 도구를 우선하고 웹 조작은 설치된 브라우저 도구를 사용한다.
4. 실패 원인을 수정하고 영향을 받는 검사를 다시 실행한다. 테스트 삭제나 기준 완화로 실패를 숨기지 않는다.
5. 대상 revision·환경·명령·결과·증거 경로를 기록하고 PASS/FAIL/NOT_RUN/UNKNOWN을 구분한다.

### 받아야 할 결과

- 수용 기준별 확인 결과
- 테스트 로그·화면·출력 등 필요한 증거
- 기존 실패와 새 실패, 미검증 환경의 구분

### 다음 단계로 가는 기준

필수 수용 기준을 확인했으면 08로 간다. 미실행 검사가 있으면 그 이유와 영향을 남기고 완료 범위를 제한한다.

**그대로 사용할 요청문**

```
수용 기준별로 실제 검증을 실행해줘.
정상 흐름과 관련 오류·재시도 흐름을 확인하고,
FAIL은 원인을 수정한 뒤 해당 검사를 다시 실행해줘.
각 결과에 대상 revision, 환경, 명령, 관찰 결과와 증거를 남겨줘.
실행하지 못한 검사는 NOT_RUN, 판단 근거가 부족하면 UNKNOWN으로 구분해줘.
```

**예시**

사진 업로드 후 새로고침해도 결과를 조회할 수 있는지 확인한다. 인증 화면으로 리다이렉트된 것만 확인했다면 로그인 이후 기능과 운영 화면은 미검증이다.

**막히면** 종료 코드 0·HTTP 200·설정 저장을 사용자 흐름 성공으로 확대하지 않는다. 브라우저마다 로그인 상태가 같다고 가정하지 말고 실제로 확인한 세션을 적는다.

OCR·검색·생성·변환은 실제 대표 입력과 독립된 기대 결과로 품질도 확인합니다. 제작 예제나 정답 주입으로 확인한 화면 흐름을 실제 입력 정확도로 확대하지 않습니다. 선택·스크롤은 긴 본문·모바일에서 대상이 실제로 보이는지 확인합니다.

# 08 검토하고 변경을 통합한다

요구사항·변경 내용·검증 결과가 서로 맞는지 검토하고 요청된 전달 단계까지 진행한다.

**작업 위치** Codex · 적용되는 외부 리뷰 도구 · GitHub PR·CI

### 사용자가 준비할 것

커밋·PR·통합이 이번 요청에 포함됐는지 명확히 한다. 이미 허용한 범위는 다시 승인하지 않아도 된다.

### AI가 수행할 순서

1. 관련 diff를 수용 기준과 비교하고 범위 밖 변경·비밀값·누락된 검사 여부를 확인한다.
2. 다중 파일 새 기능·구조 변경 또는 재현 가능한 버그라면 완료/커밋 직전에 외부 리뷰를 한 번 수행한다. 현재 외부 리뷰 지침·중앙 설정을 읽고 실제 반환 모델을 확인한다.
3. 지적은 근거로 수용 또는 기각한다. 수정한 부분은 관련 검사를 재실행한다. 단순 문서·설정 변경에는 외부 호출을 붙이지 않는다.
4. 허용 범위에서 작업 관련 변경만 stage하고 staged diff를 확인해 커밋·PR을 만든다.
5. 필수 CI·리뷰를 해당 revision에서 확인하고 허용된 merge까지 진행한다. 최신 변경 이후의 결과인지 확인한다.

### 받아야 할 결과

- 변경 목적과 실제 검증을 담은 PR 또는 완료 보고
- 실제 리뷰 결과와 미해결 사항
- 요청 범위에 따른 커밋 SHA·PR·통합 상태

### 다음 단계로 가는 기준

필수 검사·리뷰와 요청된 통합이 확인되면 G3 통과. PR 생성만 했거나 CI가 진행 중이면 그 상태로 기록한다.

**그대로 사용할 요청문**

```
현재 변경을 요구사항과 대조하고 적용 지침에 필요한 완료 경계 리뷰를 수행해줘.
리뷰 지적을 근거로 처리하고 수정분을 검증해줘.
이번 작업에 해당하는 변경만 커밋·PR로 전달해줘.
필수 CI와 허용된 통합까지 확인하고 실제 SHA·PR·결과를 보고해줘.
```

**예시**

“업로드 재시도 추가 / 중복 요청 재현 및 회귀 검사 통과 / 배포 미실행”처럼 PR에 최종 변경을 설명한다. 요청한 모델과 다른 리뷰 응답은 동일 리뷰의 통과로 처리하지 않는다.

**막히면** 리뷰 호출 실패는 미완료다. 필수 검사를 실행할 수 없으면 결과와 필요한 조건을 남긴다. 강제 push나 보호 규칙 우회는 일반적인 통합 요청에 포함되지 않는다.

이슈·PR·해당 SHA의 CI와 리뷰를 연결하고 원격 내용을 다시 읽어 저장을 확인한다. 배포가 수용 기준인 이슈는 merge만으로 자동 종료하지 않는다.

# 09 배포 준비와 실제 배포를 확인한다

배포 가능 상태, 실제 배포 완료, 사용자 기능 확인을 각각 구분한다.

**작업 위치** Codex · 기존 CI/CD · 지정된 실행 환경

### 사용자가 준비할 것

대상 환경과 이번 요청이 배포 준비인지 실제 배포인지 알려준다. 새 비용·운영 데이터 변경 등 기존 권한 밖의 결정을 정한다.

### AI가 수행할 순서

1. 설정·의존성·배포 방법·필요한 데이터 변경과 기존 롤백 방법을 구체화한다.
2. 실제 호스트·프로젝트·Docker context 등 실행 대상을 확인한다. 예전 주소나 기본 context만 믿지 않는다.
3. “배포 준비” 요청이면 검토 가능한 변경·실행 절차·롤백 조건까지 준비해 전달한다.
4. 실제 배포가 허용됐으면 기존 CI/CD 경로로 진행하고 완료 결과·대상 revision을 확인한다. 권한 밖의 마지막 행동만 구체적인 결과를 제시한 뒤 확인한다.
5. 배포 후 상태 점검과 관련 실제 사용자/API 흐름을 확인한다. 실패 시 승인 범위의 복구 또는 롤백 절차를 따른다.

### 받아야 할 결과

- 대상 환경·배포 revision·실행 결과
- 핵심 사용자/API 흐름의 확인 결과
- 롤백 방법·남은 제한·운영 확인 상태

### 다음 단계로 가는 기준

실제 환경에서 요청한 동작을 확인하면 G4 통과. 배포 준비만 요청했다면 그 결과를 완료로 전달하고 운영 확인은 NOT_RUN으로 남긴다.

**그대로 사용할 요청문**

```
[환경·서비스]에 이번 변경을 배포하고 실제 동작까지 확인해줘.
대상과 배포 revision, 기존 CI/CD·롤백 경로를 먼저 확인해줘.
이미 허용된 범위에서 진행하고 새로운 권한이 필요한 마지막 행동만 알려줘.
배포 완료와 실제 사용자/API 흐름의 확인 결과를 각각 보고해줘.
```

**예시**

CI가 성공해 이미지가 만들어진 상태, 서버에 그 이미지가 올라간 상태, 사용자가 업로드·조회에 성공한 상태는 서로 다른 증거다.

**막히면** 운영 인증이나 환경 접근이 없으면 “배포 상태 확인 / 사용자 흐름 미검증”처럼 정확히 제한한다. localhost의 성공으로 운영 성공을 대신하지 않는다.

# 10 결과를 기록하고 다음 작업을 잇는다

다음 채팅이 전체 대화를 다시 읽지 않고 현재 파일과 결과를 대조해 이어갈 수 있게 한다.

**작업 위치** Codex · 기준 GitHub 이슈/Project·PR · STATE 요약 · 같은 프로젝트의 문서로 결과 정리

### 사용자가 준비할 것

이번 결과가 실제 목적에 맞는지 확인하고 다음 우선순위를 알려준다. 이어서 진행할 때는 프로젝트 경로와 남은 목표를 전달한다.

### AI가 수행할 순서

1. 실제 구현·검증·리뷰·통합·배포 상태를 나눠 기록한다.
2. branch/revision·미커밋 변경·증거 위치·남은 문제·실패한 접근과 다음 행동을 짧게 남긴다.
3. 다음 채팅에서 상태 기록과 현재 Git·코드·필요한 CI를 대조하고 남은 단계부터 재개한다.
4. 반복 개선은 구체적인 실패와 효과가 확인됐을 때 제안한다. 전역 스킬·설정·메모리를 임의로 수정하지 않는다.
5. 예약 점검은 사용자가 요청했을 때 안정된 절차를 대상으로 구성한다. 변화가 없거나 같은 차단이면 조용히 종료하고 의미 있는 변화만 알리도록 한다.

### 받아야 할 결과

- 짧은 현재 상태와 증거 링크
- 다음 실행 가능한 작업 하나
- 재개 요청문 또는 필요한 인수인계

### 다음 단계로 가는 기준

다음 사람이 작업 위치·남은 일·확인 방법을 알 수 있으면 이번 단위가 끝난다. 미완료 항목은 완료로 바꾸지 않고 이어간다.

**그대로 사용할 요청문**

```
$gon-project-lifecycle
이 프로젝트를 이어서 진행해줘.
기준 GitHub 이슈·Project·PR/CI를 다시 읽고 로컬 요약, 현재 경로·브랜치·변경과 대조해줘.
남은 것이 구현, 실패 해결, 리뷰, 배포 중 무엇인지 정하고 그 단계부터 진행해줘.
이전 작업을 다시 초기화하지 말고, 완료하면 실제 결과와 다음 행동을 갱신해줘.
```

**예시**

“업로드·조회 PASS, 재시도 PASS, PR 통합 완료, 운영 배포 NOT_RUN, 다음은 OCR 연결”이면 다음 작업은 업로드 화면을 다시 만드는 일이 아니다.

**막히면** 기록과 코드가 다르면 현재 관찰된 사실을 기준으로 수정한다. 오래됐다는 이유만으로 무조건 전체 테스트를 반복하지 않고 변경·실패·근거가 사라진 부분을 확인한다.

로컬 PLAN/STATE·메모리는 기준 GitHub URL과 마지막 확인 시점의 요약이다. 원격 재조회가 되지 않은 기록은 pending_sync로 남기고 저장 완료로 바꾸지 않는다.

외부 환경이 막힌 검증과 실제 의존 작업만 보류하고, 독립적으로 허용된 작업은 계속합니다. 새 역할·긴 이전 맥락에는 작은 계약을 가진 새 세션을 사용합니다. 모델 계열과 명시 선택을 보존하면서 작업 난이도·영향에 맞는 effort를 고릅니다.

실행 매뉴얼

# 기획 세션에서 개발 세션으로 문서 인계

중요한 결정을 접근 가능한 문서에 남기고, 받는 쪽이 실제로 읽었는지 확인한다.

**기획 A · 같은 세션**
P1~P4 문답·근거·결정

→

**P5 최종 문서**
PROJECT-BRIEF.md + STATE.md

→

**개발 A · 새 세션**
읽기 · 요약 · 첫 작업 확인

기획 기록은 `docs/research/01-problem.md`, `02-research-plan.md`, `03-evidence.md`, `04-decision.md`로 이어집니다. P5는 대화를 통째로 붙이지 않고 확정한 요구·근거·완료 기준·미정·첫 작업을 `docs/PROJECT-BRIEF.md`에 정리합니다.

예: 사용자 답 U01 → 조사 Q01 → 근거 S01 또는 사용자 확인 U01 → 결정 D01 → 요구 R01 → 확인 기준 AC01. 실제 근거가 없는 칸은 미확인으로 남깁니다. '저장했다'는 응답 뒤 파일을 다시 열어 경로와 내용을 확인합니다.

**새 개발 세션에 보낼 요청**

현재 프로젝트의 docs/PROJECT-BRIEF.md와 docs/STATE.md를 실제로 읽어줘.
목표·첫 범위·제외·완료 기준·미정·첫 작업을 요약해줘.
기획 문서의 승인 상태와 현재 파일이 일치하는지 확인해줘.
자료가 충돌하면 내게 차이를 알려줘. 기술이 미정이면 비교한 뒤 내 선택을 기다려.
아직 구현하지 말고 인계 요약부터 보여줘.

## 세션 선택

P1~P5는 같은 기획 대화, 합의한 첫 출시 범위는 같은 개발 대화로 이어갑니다. 예제 R01~R05는 그 안에서 작은 구현·확인 단위로 순환하고, 첫 범위가 끝난 뒤 독립된 다음 범위는 STATE를 저장하고 새 세션을 엽니다. P5 확정 뒤 새 개발 세션은 권장 방식이며 강제 기능은 아닙니다. 중단한 작업은 해당 폴더의 쉘에서 `codex --yolo resume`로 이름·폴더·시각을 확인해 선택합니다. `/new`는 새 대화, `/compact`는 현재 대화 요약, fork는 대화의 갈래입니다. 어느 것도 프로젝트 파일을 자동 복제하지 않습니다.

Herdr 앱 창을 닫아도 core가 살아 있으면 실행이 유지됩니다. pane/tab을 닫거나 core가 종료되면 해당 실행은 끝납니다. 재부팅 후 Codex는 resume, 서버는 실행 명령으로 다시 시작합니다. 저장된 대화와 저장된 파일은 별개이므로 마칠 때 STATE에 세션 이름·현재 상태·다음 작업을 남깁니다.

## 개발 중의 기준 문서

PROJECT-BRIEF는 기획 출발점의 기록입니다. 개발 중 PRODUCT·ARCHITECTURE 등이 필요하면 기존 문서를 읽고 연결합니다. 범위를 바꿀 때 현재 기준과 변경 이유를 STATE에 연결하며 서로 다른 요구를 방치하지 않습니다. 기존 AEOS 계약은 우선합니다.

# 문서와 완료 증거를 관리하는 방법

프로젝트 기준은 저장소 문서로, 현재 판단은 실행 증거로 유지한다.

GitHub를 관리 기록의 기준으로 사용한다. 목적·구조·ADR·계약은 저장소 파일, 작업·상태는 Issues/Projects, 구현·검증은 PR/Actions, 전달은 Release/배포에 연결한다. Discussions·Wiki·Security·Packages/Pages·보호 설정 등 각 영역의 적용 또는 불가 사유와 대체 기록은 관리 문서에 남긴다.

| 일반 프로젝트 | 무엇을 담는가 | 언제 갱신하는가 |
|---|---|---|
| **AGENTS.md** | 필수 규칙·진입 문서·검증·리뷰 조건 | 규칙이나 작업 진입점이 바뀔 때 |
| **README.md** | 설치·실행·검사·운영 방법 | 실제 사용 방법이 바뀔 때 |
| **docs/PRODUCT.md** | 사용자·문제·범위·수용 기준 | 제품 결정이 바뀔 때 |
| **docs/ARCHITECTURE.md** | 구성·데이터·인터페이스·선택 이유 | 구조 결정이 바뀔 때 |
| **docs/PLAN.md** | 기준 Issues/Project·Milestone 링크와 다음 작업 요약 | 우선순위나 남은 작업이 바뀔 때 |
| **docs/STATE.md** | 원격 기준 URL·재조회 시점·실제 결과와 다음 행동 요약 | 의미 있는 결과가 생겼을 때 |

기존에 SPEC·STATUS·이슈 등 같은 목적의 기록이 있다면 그 경로를 유지합니다. 작은 작업에 위 파일을 모두 만들지 않습니다. STATE에는 기준 GitHub 기록의 핵심 상태와 링크만 적습니다. 다른 추적기가 지정됐다면 이전하지 않고 예외·연결을 명시합니다.

| **구현** | **검증** | **통합** | **배포** | **운영 확인** |
|---|---|---|---|---|
| 동작·diff | 대상·실행 결과 | SHA·CI·merge | 환경·revision | 실제 사용자 흐름 |

| 상태 | 의미 | 예시 |
|---|---|---|
| **PASS** | 직접 실행·관찰해 기준 충족 | 해당 revision에서 정상 업로드·조회 확인 |
| **FAIL** | 실행했고 기준 미충족 | 중복 재시도에서 동일 작업이 두 번 생성 |
| **NOT_RUN** | 실행하지 않음. 이유·영향 기록 | 운영 인증 부재로 로그인 이후 흐름 미실행 |
| **UNKNOWN** | 현재 관찰만으로 판단 불가 | 배포 요청은 접수됐지만 완료 상태 미확인 |

**STATE에 남길 최소 기록 틀**

확인 시점 / 작업 경로 / 브랜치 / revision / 미커밋 변경:
기준 GitHub Issue / Project / PR·CI / Release·배포 URL:
원격 재조회 시점 / 동기화 상태(synced·pending_sync):
이번 목표와 범위:
구현된 동작:
검증: 대상 · 환경 · 명령/행동 · 결과 · 증거
리뷰 / PR / CI / 통합:
배포 대상 · revision · 실제 흐름 확인:
남은 문제와 미정:
다음 행동과 담당:
현재 허용된 작업 범위:

비밀값과 원문 전체를 넣는 기록장이 아닙니다. 재개에 필요한 사실과 증거 위치를 적습니다. 미래에 실행할 검사 결과를 미리 PASS로 채우지 않습니다.

# 기존 AI Engineering OS를 사용하는 경우

이미 .ai 계약을 사용하거나 템플릿 도입을 선택했다면 같은 Core를 유지한다. 일반 문서 구조로 다시 초기화하지 않는다.

| 내용 | 일반 프로젝트 | AEOS |
|---|---|---|
| **제품 목적·범위** | docs/PRODUCT.md 또는 기존 명세 | PROJECT.md |
| **행동·검증 규칙** | AGENTS.md | AGENTS.md |
| **구조·인터페이스** | docs/ARCHITECTURE.md | ARCHITECT.md |
| **작업 입력** | PLAN/이슈와 관련 문서 | TASK.md + CONTEXT.md |
| **구현·검증 결과** | STATE/이슈/PR와 증거 | RESULT.md + EVAL.md |
| **검토·수용 판단** | 실제 리뷰·PR 승인 기록 | REVIEW.md + DECISION.md |

### 첫 작업의 실제 순서

1. 자신의 프로젝트 저장소에서 README·PROJECT·AGENTS·ARCHITECT를 읽고 제품 목적에 맞춘다. 템플릿 원본에 제품 코드를 작성하지 않는다.
2. 도구를 쓸 경우 현재 scripts/aeos.py와 의존 파일, 도움말을 확인한다. 수동 문서 사용은 별도 registry 승격 없이 가능하다.
3. 사용하지 않은 작업 ID로 여섯 계약의 틀을 만든다. 처음에는 TASK와 CONTEXT만 채운다.
4. 현재 AI가 계약을 읽고 구현·검증한 결과를 RESULT와 EVAL에 기록한다.
5. REVIEW와 최종 DECISION은 실제 독립 검토자 또는 사람이 담당한다. 실제로 없는 승인을 꾸며 쓰지 않는다.
6. 요청된 커밋·PR·CI·통합·배포를 거쳐 다음 작업으로 간다.

**AEOS 저장소 루트에서 사용할 명령 형식**

```
python3 scripts/aeos.py --help
python3 scripts/aeos.py validate
python3 scripts/aeos.py new-task TASK-001

# 선택 사항: 작업 입력을 초안 문서로 묶기
python3 scripts/aeos.py packet TASK-001 --runtime standalone --allow-unverified
```

Python 3.11+와 현재 CLI 의존성이 필요합니다. TASK-001은 예시이며 기존 ID가 있으면 새 ID를 고르세요. 실제 프로젝트의 도움말이 이 예시보다 우선합니다.

| 성공 표시 | 실제로 의미하는 것 |
|---|---|
| **validate PASS** | registry·이력·증거 구조가 일관됨. 제품 테스트나 모델 접속 검사가 아님. |
| **new-task 성공** | 작업 문서 6개 생성. 내용 완성이나 구현 완료가 아님. |
| **DRAFT ONLY packet** | 입력 문서를 표준 출력으로 묶음. AI 호출·현재 모델 변경·후보 모델 승격을 하지 않음. |
| **ACCEPT** | 수용 판단 기록. 별도의 배포나 외부 메시지 전송 권한을 추가하지 않음. |

Standalone과 Orca는 같은 계약을 사용할 수 있습니다. Orca 워커 실행은 별도 요청과 현재 런타임 가이드가 있을 때 연결합니다. 기본 시작을 위해 병렬 워커나 모델 qualification을 먼저 구축할 필요는 없습니다.

근거: gon-project-lifecycle/references/aeos.md와 기존 템플릿의 docs/01-getting-started.md. 주가 예측 예제는 선택 참고 자료이며 제품 기능이나 필수 도입 요소가 아닙니다.

# 프로젝트 유형별로 완료 기준을 바꾼다

진행 순서는 공유하되 “잘 동작한다”를 판단하는 증거는 제품 유형에 맞춘다.

| 유형 | 작게 시작할 기능 | 완료를 확인할 핵심 |
|---|---|---|
| **웹·앱** | 입력 → API → 저장 → 결과 확인한 흐름 | 요구 기능, UI·오류·빈 상태, 관련 API·DB 계약, 필요한 접근성·화면 크기·실제 흐름 |
| **CLI·라이브러리** | 입력 한 종류를 처리하고 결과·오류 반환 | 설치·실행 재현성, 종료 코드·출력 계약, 오류 입력, 호환성, 배포 패키지의 실제 사용 |
| **AI Skill** | 대표 요청 하나의 절차와 결과 | 명시 호출, 적합/부적합 상황 구분, 기존 자료 보존, 반복 결과, 로더 발견과 실제 적용의 구분 |
| **AI Agent** | 도구 하나로 목표 하나 수행 | 작업 성공, 잘못된 도구 호출·권한 초과 방지, 실패 복구, 필요한 비용·시간·추적 |
| **MCP·Plugin** | 도구 하나의 입력과 결과 계약 | 스키마, 인증·권한, 정상·실패 응답, 실제 서비스 연동과 민감정보 보호 |

### 시험지 OCR로 따라가는 예시

| 작업 순서 | 사용자에게 보이는 결과 | 검증할 것 |
|---|---|---|
| **A · 업로드와 조회** | 사진을 올리면 저장된 항목을 다시 연다. | 정상 형식, 잘못된 형식, 저장·조회 일치 |
| **B · OCR 연결** | 실제 사진에서 인식한 글자를 확인한다. | 실제 연동, 대표 자료의 품질, 판독 불가 처리 |
| **C · 수정과 재시도** | 인식 결과를 고치고 실패 작업을 다시 실행한다. | 수정 보존, 중복 작업 방지, 실패 후 복구 |
| **D · 승인된 후속 기능** | 해설·내보내기 등 확정된 기능을 추가한다. | 제품 목적에 맞는 품질과 오류·한계 표시 |

각 작업에서 05–08을 반복하고, 릴리스 범위가 준비되면 09로 갑니다. OCR 정확도나 AI 해설 품질은 “API 응답을 받았다”로 판정하지 말고 대표 자료와 합의한 기준으로 평가합니다.

### 모델·위임·자동화의 기본 운영

- 현재 명시된 모델과 추론 수준, 적용 정책을 보존합니다. 이전 문서의 모델 추천표만으로 현재 세션을 바꾸지 않습니다.
- model-router 같은 별도 스킬은 실제 설치·활성화·호출 정책을 확인한 후 요청된 범위에서 사용합니다. 스킬을 읽는 것이 모델 실행이나 교체의 증거는 아닙니다.
- 위임은 사용자 또는 적용 지침이 요청한 독립 작업에 한정합니다. 소유 파일과 통합 책임을 정하고 결과를 주 작업에서 확인합니다.
- 수동으로 안정화된 점검만 요청에 따라 예약합니다. 주기·대상·권한·알림 조건과 실제 실행 결과를 확인합니다.

반복 절차를 스킬로 정리한 뒤 안정된 작업을 예약하는 원칙은 OpenAI 공식 안내와 일치합니다. 이 매뉴얼은 예약 작업을 생성하지 않습니다.

# 문제가 생겼을 때의 복구 순서

성공으로 간주할 근거가 없는 지점에서 원인을 좁히고, 가능한 작업은 계속한다.

| 상황 | 먼저 확인 | 다음 행동 |
|---|---|---|
| **스킬이 보이지 않음** | 현재 세션 목록·실제 경로·활성 설정·로더 오류 | 이름만 추측해 적용하지 말고 설치·발견 상태를 확인. 다른 환경이면 별도 확인. |
| **기획 자료를 개발 세션이 못 읽음** | 링크·원문·저장소 접근 가능 여부 | 확정 범위·계약·검증 기준을 파일 또는 접근 가능한 원문으로 전달. |
| **다른 파일이 보임** | 실제 경로·브랜치·worktree·Local/Cloud | 올바른 작업 대상을 확인. 대화 동기화를 파일 복사로 이해하지 않기. |
| **문서 틀만 생성됨** | 내용의 실제 값과 명령·링크 | 구현 요청이면 내용 연결 후 제품 구현·검증까지 계속. |
| **테스트 실패** | 착수 전 기준선과 이번 diff | 기존 실패와 회귀를 구분하고 관련 원인을 수정. 동일 가설 반복 금지. |
| **외부 리뷰 실패** | 호출 결과·반환 모델·지침 | 미완료로 기록하고 필요한 조건 해결. 자동 모델 대체로 통과 처리 금지. |
| **CI 성공, 화면은 실패** | 해당 revision·인증·실제 환경 | 사용자 흐름을 재현. YAML·HTTP 응답만으로 완료 판정 금지. |
| **AEOS 생성기가 중단** | 부분 .ai·aeos.py·계약·registry 의존성 | 일반 생성기를 강제로 적용하지 말고 기존 체계를 확인. 가능한 수동 문서 작업 진행. |
| **배포 권한이 부족** | 현재 요청·대상·남은 권한 | diff·PR·검증·배포 계획을 준비한 뒤 마지막 필요한 권한만 확인. |

**차단을 풀기 위한 요청문**

현재 막힌 지점을 실제 증거로 정리해줘.
관찰한 사실, 실패한 가설, 현재 완료된 범위,
독립적으로 진행할 수 있는 일, 다음에 필요한 조건을 구분해줘.
같은 시도를 반복하거나 미완료를 통과로 표시하지 말아줘.

## 완료 보고를 읽는 다섯 가지 질문

- 요청한 사용자 행동이 실제로 구현됐는가?
- 어떤 revision과 환경에서 무엇을 검사했는가?
- 미실행·실패·불확실한 항목이 구분됐는가?
- 커밋·PR·통합·배포·운영 확인 중 어디까지 도달했는가?
- 다음 사람이 바로 할 행동이 한 가지 이상 명확한가?

# 앞으로 매번 사용하는 짧은 운영 절차

사용자는 목표와 중요한 선택을 맡고, AI는 현재 상태 확인부터 결과 기록까지 이어간다.

**시작할 때**

**경로 + 목표 + 범위 + 완료 기준**

새 프로젝트면 첫 실행 요청문. 기존 작업이면 재개 요청문을 쓴다.

**진행 중**

**실제 결과를 보고 방향 조정**

새 요구는 이번 범위 또는 후속 작업으로 명시한다. 같은 문제는 같은 채팅에서 이어간다.

**마칠 때**

**동작 + 검증 + 상태 + 다음 행동**

현재 구현·검증·배포 수준을 확인하고 결과를 남긴다.

**대부분의 작업에 쓰는 기본 요청문**

$gon-project-lifecycle
[프로젝트 경로]의 현재 상태를 확인하고
[이번 목표]를 [완료 조건]까지 진행해줘.
기존 지침과 변경을 보존하고 필요한 검사·완료 경계 리뷰를 수행해줘.
실제 결과와 남은 제한, 다음 행동을 기록해줘.

**작업을 마치거나 다른 채팅으로 넘어갈 때**

이번 작업을 여기까지 정리해줘.
실제 완료한 동작, 실행한 검증과 증거,
리뷰·PR·배포 상태, 남은 문제와 다음 작업을 기존 상태 문서에 남겨줘.
다음 채팅에서 바로 이어갈 짧은 요청문도 작성해줘.

## 첫 적용의 성공 기준

처음에는 작은 기능 하나로 확인하세요. 스킬을 실제로 읽고, 기존 문서를 보존하며, 구현 후 검증 결과를 남기고, 다음 채팅에서 그 결과를 이어갈 수 있으면 첫 사용 흐름이 검증된 것입니다. 그다음 적용 범위를 늘리면 됩니다.

**이 매뉴얼을 바꾸는 시점**
실제 작업에서 반복된 누락이나 실패가 확인됐을 때, 원인·바꿀 절차·확인 방법을 정리해 수정합니다. 새 도구가 생겼다는 이유만으로 모든 프로젝트의 규칙을 자동 교체하지 않습니다.

재생성 · 실행 양식 · 검증

# GitHub 원본으로 전체 안내서 다시 만들기

관리 매뉴얼, Herdr·Orca 안내서, 프로젝트 실행 양식과 OCR 예시를 같은 저장소에서 관리한다.

전체 재생성 절차와 AI 요청문에서 환경을 준비한 뒤 아래 명령을 실행합니다.

```
npm run build:all
npm test
npm run verify:all
```

- Herdr 단계별 안내서 · Orca 단계별 안내서
- 아이디어 입력용 실행 양식 · 시험지 OCR 프로젝트 문서 예시

편집 JSON·생성기·검사 도구를 수정하고 HTML/PDF만 고치지 않습니다. 공통 P1~P5는 전체 빌드에서 Herdr 안내에 동기화합니다. 흐름도를 바꾸면 별도 Archify 검증이 필요합니다.

> **검증 범위**
> 문서 재생성·브라우저 회귀·PDF 검사는 실제 Herdr/Orca 협업이나 제품 OCR·배포 성공이 아닙니다. 실행 양식은 주 모델 내부에서 앱/core 접근을 먼저 확인하고, 미확인 모델·사용자 승인·수용 검사를 성공으로 꾸미지 않습니다.

# 적용 기준과 원본 자료

2026년 10월 31일 프로젝트 대화·산출물·설치된 스킬을 통합한 매뉴얼이다. 매뉴얼은 gon-development-os, 실행 스킬은 gon-skills에서 관리한다.

### 대화에서 정리된 내용과 반영 위치

| 프로젝트 대화 | 매뉴얼에 반영한 내용 |
|---|---|
| **1.ChatGPT Codex Work 비교 Guide** | 작업 방식·화면·실행 장소·모델 구분, 인계, 파일·세션 경계, 브라우저와 예약 작업 |
| **2.개발 워크플로우 정리** | 통합 개발 흐름, 저장소 중심 기록, 작은 기능 반복, 유형별 완료 기준 |
| **3.Gon AI Development OS 표준 확인 경로 찾기** | 개발 체계·보일러플레이트·제안된 패키지 이름의 구분, 실제 스킬 중심 사용 |
| **gon-project-lifecycle 스킬 생성** | 시작·재개·검증·전달 절차, 문서 생성기, AEOS 연결, 검증 범위 |
| **스킬 저장 위치가 다른 이유 찾기** | 현재 두 개인 스킬 경로와 과거 설치 경로의 구분 |

긴 「1.ChatGPT Codex Work 비교 Guide」 응답은 대화 조회 도구의 메시지 길이 제한으로 끝부분이 일부 생략됐습니다. 실행 절차는 현재 설치된 스킬 원문과 대조했습니다. 원본 대화의 모든 문장을 그대로 재수록한 문서는 아닙니다.

### 현재 실행 기준으로 읽은 원본

- gon-project-lifecycle/SKILL.md — 공통 진입점과 단계별 연결
- project-start.md · work-cycle.md · delivery.md · aeos.md — 상세 절차
- gon-web-engineering/SKILL.md — 웹 구현 기본안과 기존 스택 보존
- AEOS 처음 시작하기 · AEOS 설계 방향 기록 — 기존 계약 사용법과 역사적 결정
- 스킬 생성·이동 검증 기록 · 기존 배포용 ZIP

### 이전 안내에서 현재 기준으로 정리한 항목

| 이전 내용 | 이 매뉴얼의 기준 |
|---|---|
| **Gon 표준과 ai-engineering-os를 같은 것으로 설명** | 통합 개발 방식과 기존 보일러플레이트를 구분한다. |
| **스킬을 아직 만들어야 한다는 초기 제안** | gon-project-lifecycle·gon-web-engineering은 현재 세션에서 읽을 수 있는 설치 스킬이다. |
| **gon-project-lifecycle의 과거 .codex/skills 경로** | 개인 설치 위치는 ~/.agents/skills/gon-project-lifecycle이고 원본은 gon-skills의 skills/gon-project-lifecycle이다. 이 매뉴얼의 skills/에는 채택한 버전의 사본을 둔다. |
| **gon-development-os 및 추가 전문 스킬 묶음** | gon-development-os는 현재 관리 저장소로 구성했다. 추가 전문 스킬과 플러그인 패키징은 여전히 확장안이다. |
| **모든 작업의 동일 10단계·고정 모델 추천표** | 규모에 맞는 단계만 적용하고, 명시 모델·현재 정책을 유지한다. |

| 이전 내용 | 이 매뉴얼의 기준 |
| --- | --- |
| **Work·로컬·플러그인의 자동 전달을 기대** | 환경별 발견·접근·실제 읽기·실행을 확인한다. |

### 검증 기록의 의미

기존 기록의 스킬 파일 14개 해시가 최초 매뉴얼 제작 시점의 설치 파일과 모두 일치함을 확인했습니다. 원래 파일과 검증 기록은 archive/2026-10-31에 보관합니다. 기존 기록은 테스트 20개, 형식·링크·로더·AEOS smoke 검사 통과와 외부 리뷰 반영을 담고 있습니다. 최초 매뉴얼 제작에서는 테스트를 재실행하지 않았습니다. 저장소 통합 시의 새 검사와 이후 결과는 evidence 및 GitHub Actions 기록으로 구분합니다. 자동 스킬 선택, 실제 제품 구현·운영 배포, 다른 OS·Orca 런타임의 발견은 별도 확인 대상입니다.

공식 참조: Skills & Plugins · 개인·프로젝트 스킬 경로 · 반복 작업과 예약. 계정별 지원·권한·메모리 설정은 작업 시 확인하며, 이 매뉴얼을 사용하기 위해 임의로 설정을 변경하지 않습니다.

기획·리서치 단계 보완(2026-11-01): OpenAI Quickstart와 프로젝트·채팅 공식 안내에서 Chat/Work 진입을 확인했다. P1~P5의 질문·조사·의사결정 순서는 Gon 운영 절차이며, 실제 조사 결과나 제품 수요 검증을 완료했다는 뜻이 아니다.

### Herdr + Codex CLI 판 · 2026-11-02

Herdr app 0.1.11의 설치 문서·키 설정·commands와 Codex CLI 0.166.0 도움말을 확인했습니다. Herdr app, tmux pane, Codex CLI 공식 명령을 대조했습니다. 실제 사용자 앱 조작·로그인·재부팅 복구는 수행하지 않았습니다. 앞의 Chat/Work 자료는 기존 개념의 출처이며 이 판의 실행 화면은 Herdr로 통일합니다.